BSP-генерация (Binary Space Partitioning)

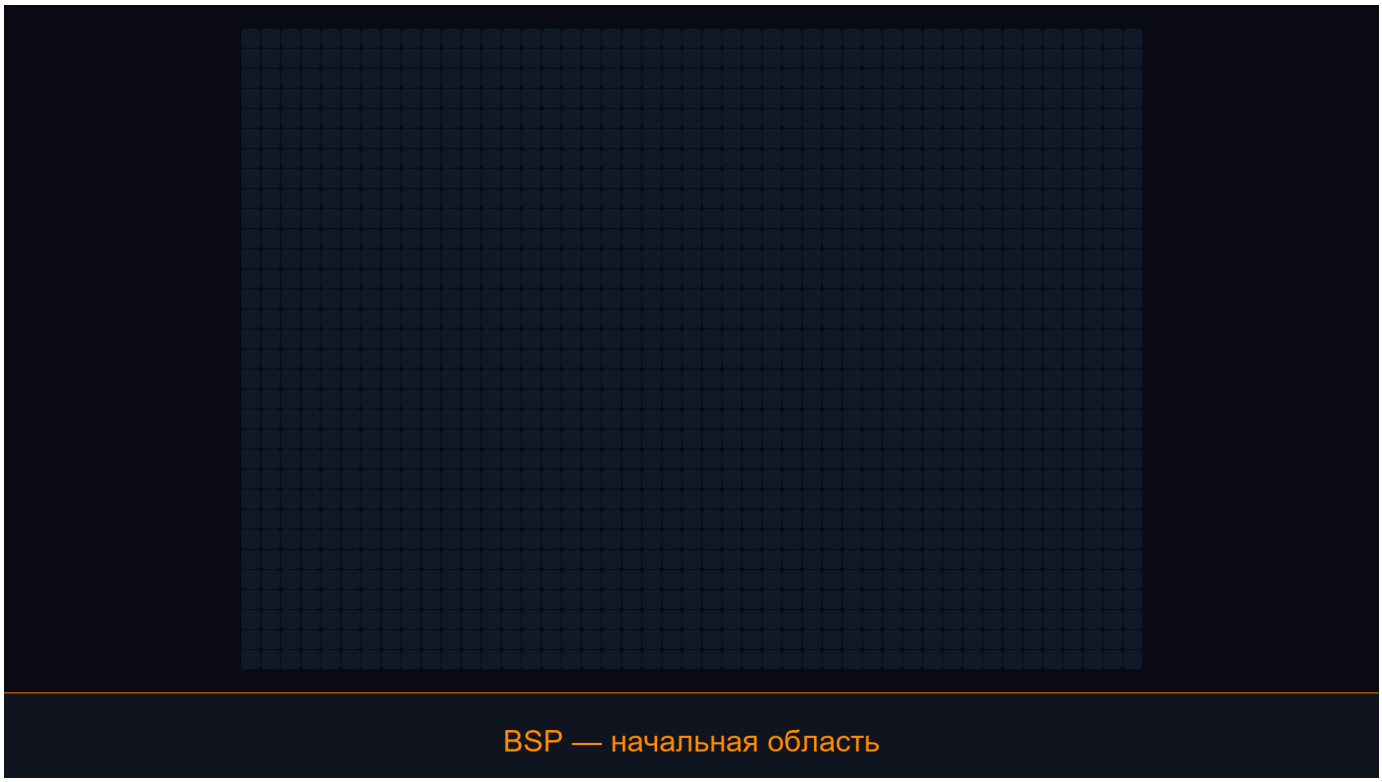
BSP-генерация (Binary Space Partitioning)

Идея

Случайное блуждание и клеточные автоматы создают пещеры. Естественные, органические, но пещеры. А что, если нам нужно подземелье? С комнатами, коридорами, дверьми? Как в классике - Binding of Isaac, Enter the Gungeon, Hades.

Для этого нужен другой подход. Нам нужно как-то **разбить пространство** на области, затем в каждой разместить комнату и соединить их коридорами. И желательно, чтобы комнаты не накладывались друг на друга и были распределены более-менее равномерно.

BSP (Binary Space Partitioning) - двоичное разбиение пространства - решает именно эту задачу. Кстати, BSP уже упоминался в контексте коллизий и разбиения пространства, где он помогал с отсечением невидимой геометрии и ускорением коллизий. Здесь же мы применим ту же идею для генерации уровней.



Принцип работы

Алгоритм состоит из трёх этапов: разбиение, размещение комнат, прокладка коридоров.

Этап 1: Разбиение пространства

Берём всю область уровня - один большой прямоугольник. Делим его пополам. Линия разреза может быть вертикальной или горизонтальной - выбираем случайно. Или, чтобы избежать слишком длинных и узких частей, разрезаем по длинной стороне - эту идею можно использовать для тонкой настройки генерации.

Получили два прямоугольника. Каждый из них делим снова. И так далее, пока части не станут достаточно маленькими - скажем, не меньше 7x7 тайлов.

Что у нас получается? Дерево. Корень - это вся область. У каждого узла - два потомка, левый и правый (или верхний и нижний). Листья дерева - это финальные области, в которых будут комнаты.

Почему именно двоичное? Потому что на каждом шаге мы делим область на **две** части. Отсюда и "Binary" в названии.

Этап 2: Размещение комнат

Теперь проходим по листьям дерева. В каждом листе создаём комнату - прямоугольник, который **меньше** области листа. Зачем меньше? Чтобы между комнатами оставалось пространство для стен и коридоров. Комната же не должна занимать всю область. А то мы так получим один большой зал.

Размер и положение комнаты внутри области выбираем случайно, но с ограничениями: минимальный размер комнаты (скажем, 3x3) и отступ от краёв области (хотя бы 1 тайл).

Этап 3: Прокладка коридоров

Осталось соединить комнаты. И тут дерево нам очень помогает. Мы не соединяем все комнаты со всеми - мы идём по дереву снизу вверх и соединяем **соседние** узлы.

Для каждого узла, у которого есть два потомка, мы берём по одной комнате из левого и правого поддерева и прокладываем между их центрами коридор. Самый простой коридор - L-образный: сначала горизонтальный отрезок, потом вертикальный (или наоборот).

Поскольку мы делаем это для каждого узла дерева, все комнаты в итоге оказываются связаны. Это следствие структуры дерева - если соединить потомков каждого узла, то всё дерево связано.

Реализация

Ок, алгоритм сложнее предыдущих - но каждая часть по отдельности простая. Разберём их по очереди. Часть покажем кодом, часть опишем словами - собирать всё в работающий генератор вы будете сами.

BSPNode - структура узла

Каждый узел хранит свою область (`x, y, width, height`), ссылки на потомков (`left, right`) и, если это лист, - комнату (`room`).

Python C#

```python
import random


class BSPNode:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.left = None
        self.right = None
        self.room = None  # (rx, ry, rw, rh)
```

Split - разбиение узла

Метод `split` делит узел на два потомка. Логика:

1. Если узел уже разбит (есть `left` или `right`) - ничего не делаем, возвращаем `false`.
2. Выбираем направление разреза: по длинной стороне, чтобы области не вытягивались в полоски. Если стороны равны - случайно.
3. Считаем максимальную позицию разреза: длина выбранной стороны минус `min_size`. Если она не больше `min_size` - область слишком мала, возвращаем `false`.
4. Выбираем позицию случайно от `min_size` до максимальной. Это гарантирует, что оба потомка будут не меньше `min_size`.
5. Создаём два дочерних узла. При горизонтальном разрезе делим по высоте, при вертикальном - по ширине. Координаты второго потомка сдвигаются на `split_pos`.

Чтобы разбить всё пространство, нужен цикл: сначала `split` для корня, потом для его потомков, потом для их потомков - `depth` итераций. На каждой итерации собираем текущие листья и пытаемся разбить каждый.

CreateRooms - размещение комнат

Рекурсивно обходим дерево. Не лист - спускаемся в потомков. Лист - создаём комнату случайного размера с отступом `padding` от краёв области. Ячейки сетки становятся полом (0).

Python C#

```python
def create_rooms(self, grid, min_room=3, padding=1):
    if self.left or self.right:
        if self.left:
            self.left.create_rooms(grid, min_room, padding)
        if self.right:
```

```
            self.right.create_rooms(grid, min_room, padding)
    else:
        max_w = self.width - padding * 2
        max_h = self.height - padding * 2
        if max_w < min_room or max_h < min_room:
            return

        rw = random.randint(min_room, max_w)
        rh = random.randint(min_room, max_h)
        rx = self.x + random.randint(padding, self.width - rw - padding)
        ry = self.y + random.randint(padding, self.height - rh - padding)

        self.room = (rx, ry, rw, rh)
        for y in range(ry, ry + rh):
            for x in range(rx, rx + rw):
                grid[y][x] = 0
```

GetRoomCenter - центр комнаты

Если узел - лист с комнатой, возвращает её центр. Иначе спускается к потомкам и берёт центр первой найденной комнаты. Нужен для прокладки коридоров.

Python C#

```
def get_room_center(self):
    if self.room:
        rx, ry, rw, rh = self.room
        return (rx + rw // 2, ry + rh // 2)
    if self.left:
        return self.left.get_room_center()
    if self.right:
        return self.right.get_room_center()
    return None
```

ConnectChildren - соединение комнат

Ключевая часть, которая обеспечивает связность подземелья. Метод `connect_children` рекурсивно обходит дерево. Для каждого узла с двумя потомками:

1. Сначала рекурсивно вызываем `connect_children` для левого и правого потомков - чтобы их внутренние комнаты тоже были соединены.
2. Берём центр комнаты из левого поддерева (через `get_room_center`) и центр из правого.
3. Если оба центра найдены - прокладываем коридор между ними.

Получается, что на каждом уровне дерева мы "сшиваем" левую и правую половину. А поскольку мы делаем это рекурсивно для всех уровней - все комнаты оказываются связаны.

CarveCorridor - прорезание коридора

L-образный коридор: сначала горизонтальная линия от одной точки до другой по оси X, потом вертикальная по оси Y. Гарантированно соединяет любые две точки.

Python C#

```python
def carve_corridor(grid, start, end):
    x1, y1 = start
    x2, y2 = end
    # L-shaped: horizontal then vertical
    for x in range(min(x1, x2), max(x1, x2) + 1):
        grid[y1][x] = 0
    for y in range(min(y1, y2), max(y1, y2) + 1):
        grid[y][x2] = 0
```

Осталось собрать: создайте сетку из единиц (стены), корневой `BSPNode` на всю область, разбейте его `depth` раз через цикл, затем вызовите `create_rooms` и `connect_children` на корне.

Результат

BSP даёт классическое подземелье: отдельные комнаты, соединённые коридорами. Каждая комната - прямоугольник. Каждый коридор - "Г"-образная линия шириной в один тайл.

Влияние параметров

- **depth** (глубина разбиения): чем больше, тем больше комнат, но каждая - меньше. При `depth=3` получается 4-8 крупных комнат. При `depth=5` - до 32 маленьких.
- **min_size** (минимальный размер области): ограничивает, насколько мелко можно дробить. Если область меньше `min_size` - разбиение прекращается.
- **min_room** (минимальный размер комнаты): влияет на то, насколько маленькими могут быть комнаты внутри областей.
- **padding** (отступ): расстояние между комнатой и краем области. Чем больше - тем больше "стен" между комнатами.

Попробуйте `bsp_generate(50, 40, depth=3)` для нескольких крупных комнат и `bsp_generate(50, 40, depth=5)` для лабиринта из мелких. Обратите внимание,

как меняется "характер" подземелья.

Гарантия связности

В отличие от клеточных автоматов, BSP **гарантирует**, что все комнаты связаны. Ну, грубо говоря - почему? Потому что на каждом уровне дерева мы соединяем левого и правого потомка коридором. А поскольку дерево - связная структура, все листья (комнаты) оказываются достижимы из любого другого листа.

Получается, отдельная валидация здесь не нужна. Это одно из главных преимуществ BSP перед другими подходами.

Вариации

Комнаты-пещеры

Прямоугольные комнаты выглядят довольно стерильно. Можно разнообразить их, применив клеточные автоматы внутри каждой комнаты. Т.е. BSP определяет расположение и связность, а автоматы - форму. Стены комнаты становятся неровными, появляются выступы, ниши.

Коридоры шире одного тайла

L-образный коридор шириной в один тайл - это минимальный вариант. Можно прорезать коридоры шириной 2-3 тайла, или даже делать их случайной ширины. Для этого достаточно в `carve_corridor` прорезать не одну линию, а несколько параллельных.

Случайность разреза

В текущей реализации мы всегда режем по длинной стороне. Это даёт более "квадратные" области. Если добавить случайность в выбор направления (например, 30% шанс разрезать по короткой стороне), комнаты будут разнообразнее по форме.

Неравномерное разбиение

Сейчас позиция разреза выбирается равномерно. Можно сместить его ближе к центру (чтобы части были более равными) или, наоборот, к краю (чтобы получить одну большую и одну маленькую комнату).

Плюсы и минусы

- **Гарантированная структура**: всегда получаются отдельные комнаты и коридоры. Никакого хаоса.
- **Гарантированная связность**: все комнаты достижимы. Валидация не нужна.
- **Контроль**: параметры дают предсказуемое влияние на результат. Глубже - больше комнат, меньше размер.
- **Расширяемость**: легко добавить дополнительную логику - типы комнат, размещение врагов/лута по областям дерева, мини-боссы в крупных комнатах.

Но:

- **"Коробочность"**: комнаты - прямоугольники, коридоры - прямые линии. Результат может выглядеть искусственно. Для исправления нужна постобработка (автоматы, шум).

- **Сложность реализации**: алгоритм посложнее случайного блуждания или клеточных автоматов. Больше кода, больше мест для ошибок.
- **Однообразие**: подземелья, сгенерированные BSP, имеют узнаваемую "сетку". Если не добавлять вариативность, игрок быстро заметит паттерн.

Что дальше?

- Случайное блуждание и клеточные автоматы - если хотите более органичные результаты, или для комбинирования с BSP.
- Шум Перлина - для ландшафта и открытых миров, совсем другая задача.
- Wave Function Collapse - генерация из правил соседства.
- L-системы - формальные грамматики для растений и фракталов.
- Пространственные структуры - ускорение коллизий через разбиение пространства.

Powered by Wiki.js